```
def fibonacci(n):
    a, b = 0, 1

    for _ in range(n):
        print(a, end=" ")
        a, b = b, a + b



try:
    n = int(input("Enter the number of terms: "))

    if n <= 0:
        raise ValueError("Please enter a positive integer.")

    print("Fibonacci series:")
    fibonacci(n)

except ValueError as e:
    print("Error:", e)
```

output :

Enter the number of terms: 7

Fibonacci series:

0 1 1 2 3 5 8

```
def job_sequencing(jobs):
    jobs.sort(key=lambda x: x[2], reverse=True)

    max_deadline = max(job[1] for job in jobs)

    slots = [None] * max_deadline

    total_profit = 0

    for job_id, deadline, profit in jobs:
        for i in range(min(deadline, max_deadline) - 1, -1, -1):
            if slots[i] is None:
                slots[i] = job_id
                total_profit += profit
                break

    return slots, total_profit



try:
    n = int(input("Enter number of jobs: "))

    if n <= 0:
        raise ValueError("Number of jobs must be positive.")

    jobs = []

    for i in range(n):
        job_id = input(f"Enter Job ID for job {i + 1}: ")
        deadline = int(input(f"Enter deadline for {job_id}: "))
        profit = int(input(f"Enter profit for {job_id}: "))
```

```
        if deadline <= 0 or profit < 0:
            raise ValueError("Deadline must be positive and profit cannot be negative.")

        jobs.append((job_id, deadline, profit))

    scheduled_jobs, total_profit = job_sequencing(jobs)

    print("\nScheduled Jobs:")
    print([job for job in scheduled_jobs if job is not None])

    print("Total Profit:", total_profit)

except ValueError as e:
    print("Error:", e)
```

output:

```
Enter number of jobs: 5
Job A: deadline = 2, profit = 100
Job B: deadline = 1, profit = 19
Job C: deadline = 2, profit = 27
Job D: deadline = 1, profit = 25
Job E: deadline = 3, profit = 15
Scheduled Jobs:
['C', 'A', 'E']
Total Profit: 142
```